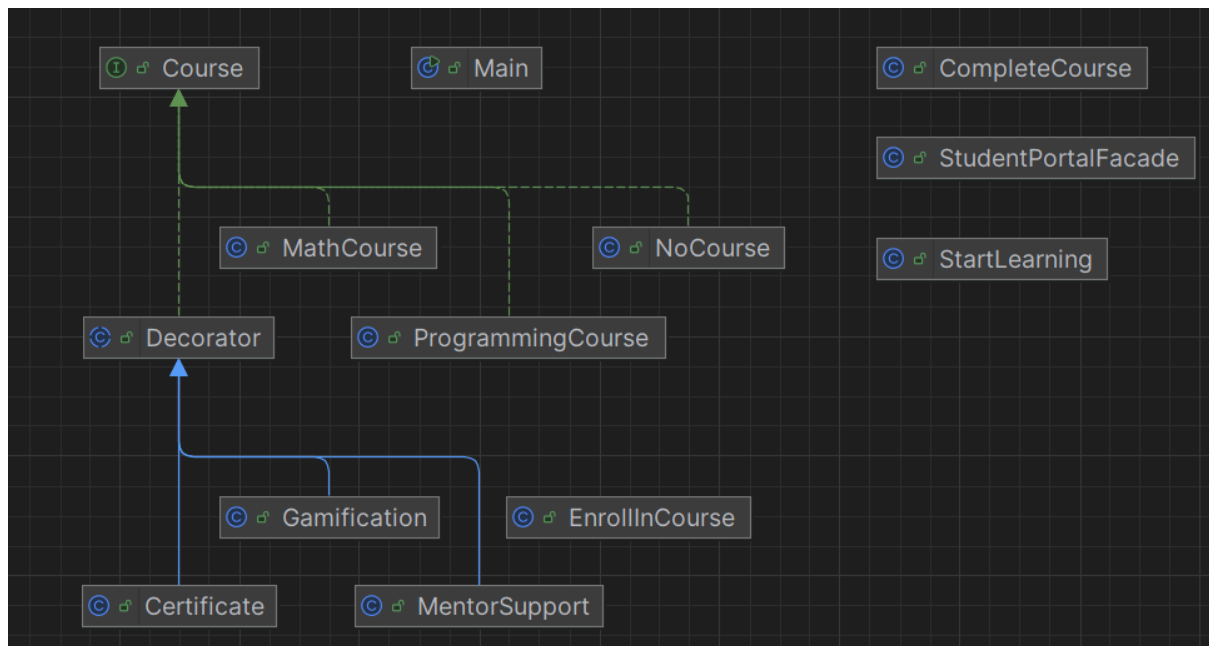
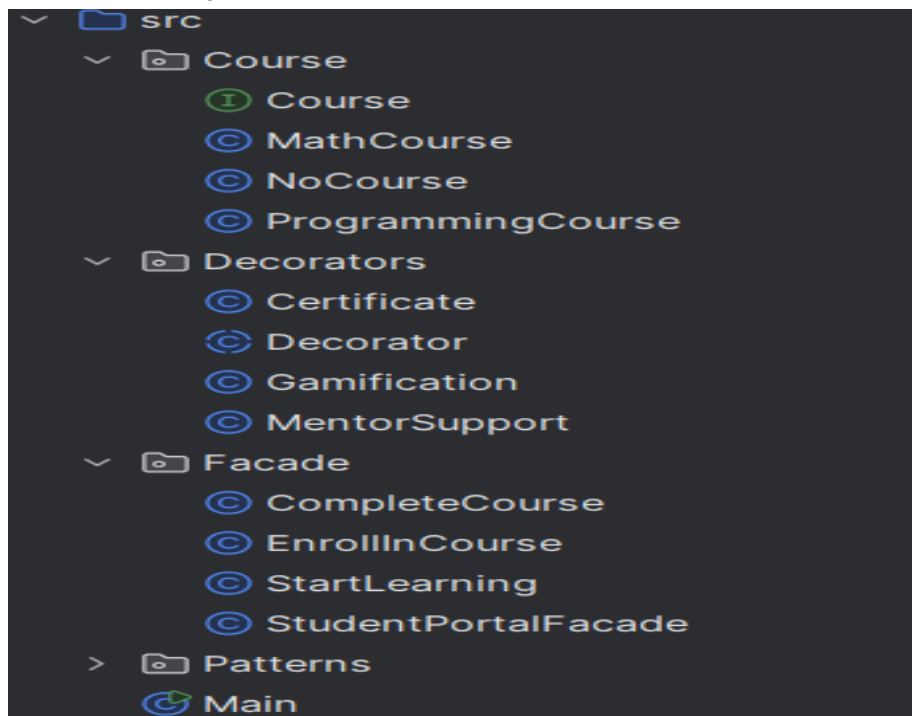


SDP ASSIGNMENT 5

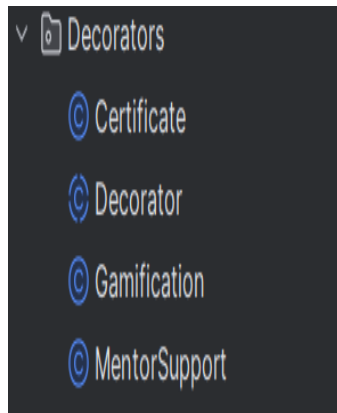
Kuanysh Beibarys SE-2413

Variant 3 LMS

Structure of Project



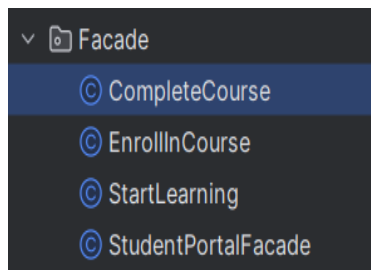
Decorator Pattern



```
1 package Decorators;
2
3 import Course.Course;
4
5 @ public abstract class Decorator implements Course { 3 usages 3 inh
6     public Course course;
7
8     public Decorator(Course course) { 3 usages 3 Gorkiy Chocolate
9         this.course = course;
10    }
11
12    @Override 6 usages 3 overrides 3 Gorkiy Chocolate
13    public void deliverContent() {
14        course.deliverContent();
15    }
16
17
18 }
```

```
1 package Decorators;
2
3 import Course.Course;
4
5 public class Certificate extends Decorator { 2 usages 3 Gorkiy Chocolate
6     public Certificate(Course course){ 1 usage 3 Gorkiy Chocolate
7         super(course);
8     }
9     @Override 6 usages 3 Gorkiy Chocolate
10    public void deliverContent() {
11        super.deliverContent();
12        System.out.println("grants certificate after completion.");
13    }
14 }
```

Facade Pattern



```
1 package Facade;
2
3 import Course.*;
4
5 public class StudentPortalFacade{ 2 usages 3 Gorkiy Chocolate
6     EnrollInCourse enrollInCourse = new EnrollInCourse(); 1 usage
7     StartLearning startLearning = new StartLearning(); 1 usage
8     CompleteCourse completeCourse = new CompleteCourse(); 1 usage
9
10    public void enrollInCourse(Course course) { no usages 3 Gorkiy Chocolate
11        enrollInCourse.enrollInCourse();
12    }
13
14    public void startLearning(Course course) { no usages 3 Gorkiy Chocolate
15        startLearning.startLearning();
16    }
17
18    public void completeCourse(Course course) { no usages 3 Gorkiy Chocolate
19        completeCourse.completeCourse();
20    }
21
22 }
```

```
1 package Facade;
2
3 public class EnrollInCourse{ 2 usages 3 Gorkiy Chocolate
4     public void enrollInCourse() { System.out.println("Enrolling in Course"); }
5
6
7
8 }
```

Main File

```
1  import Course.*;
2  import Decorators.Certificate;
3  import Decorators.Gamification;
4  import Decorators.MentorSupport;
5  import Facade.*;
6
7
8  public class Main { @ Gorkiy Chocolate
9      public static void main(String[] args) { @ Gorkiy Chocolate
10          System.out.println("Variant 3: Learning Management System (LMS)");
11          MathCourse mathCourse = new MathCourse();
12          ProgrammingCourse programmingCourse = new ProgrammingCourse();
13
14          Course student1 = new Certificate(new MentorSupport(mathCourse));
15          Course student2 = new Gamification(programmingCourse);
16          StudentPortalFacade facade = new StudentPortalFacade();
17          System.out.println("Demo");
18
19          System.out.println("Student enrolls in MathCourse with MentorSupport + Certificate");
20
21          facade.enrollInCourse(student1);
22          student1.deliverContent();
23          facade.startLearning(student1);
24          facade.completeCourse(student1);
25
26          System.out.println("-----");
27
28          System.out.println("Student enrolls in ProgrammingCourse with Gamification");
29          facade.enrollInCourse(student2);
30          student2.deliverContent();
31          facade.startLearning(student2);
32      }
```

1. Decorator Pattern

Purpose: To dynamically add extra functionality to a Course object without modifying its structure.

Base Interface: Course (method deliverContent())

Concrete Components:

MathCourse

ProgrammingCourse

Decorators:

Certificate → Grants a certificate upon completion

MentorSupport → Adds personal mentor support

Gamification → Adds points and leaderboard

2. Facade Pattern

Purpose: To provide a simplified interface for students to interact with the learning system.

Class: StudentPortalFacade

Methods:

enrollInCourse(course)

startLearning(course)

completeCourse(course)

The facade hides the complexity of managing decorated courses and coordinates the learning process through high-level commands.